Runtime speedups for bucket-partitioned CVRP

Ours vs BP-MDS on the CPU lane (cuOpt next)

Ash

Abstract

BP-MDS solves capacitated vehicle routing by cutting the plane into angular sectors around the depot, building a minimum spanning tree in each sector, sampling randomized depth-first tours, and refining with 2-opt. This note is the first scoreboard entry for our fork: keep that combinatorial structure fixed, rewrite the hot path, and measure wall time and tour cost against upstream BP-MDS. The wider goal is to beat both **BP-MDS** (CPU) and **NVIDIA cuOpt** (GPU) on time and gap-to-BKS; the GPU lane is future work. On matched CPU runs (α=2, ρ=10^4, 48 threads), solve time drops by **76.6%** at 10^6 customers and **45.6%** at 10^7, while tour cost stays essentially flat.

1 Pipeline

Figure 1 is the end-to-end flow. Only the shaded stages change in the modified solver; the accepted routes still come from MST + randomized DFS + 2-opt.

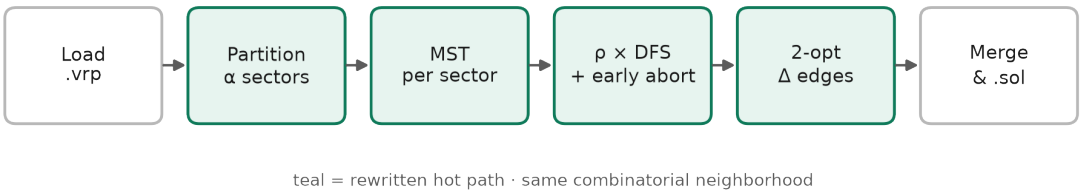


Figure 1: Solver pipeline. Teal stages are where the speedups live.

2 What changed

Hot path	Before	After	Why faster
Bucket assign	unit-vector sweeps	atan2	one angle per customer
MST	heap Prim + $\sqrt{\cdot}$	dense Prim on squared length	no heap; $\sqrt{\cdot}$ monotone
2-opt	full route rescan	$O(1)$ edge Δ	same moves, less work
DFS	per-node alloc	one buffer + early abort	less alloc; skip losers
OpenMP	critical every trial	thread-local best	less lock contention
Distance	bounds-checked	`distance_fast`	less overhead on hot path

2.1 A

ngular partition

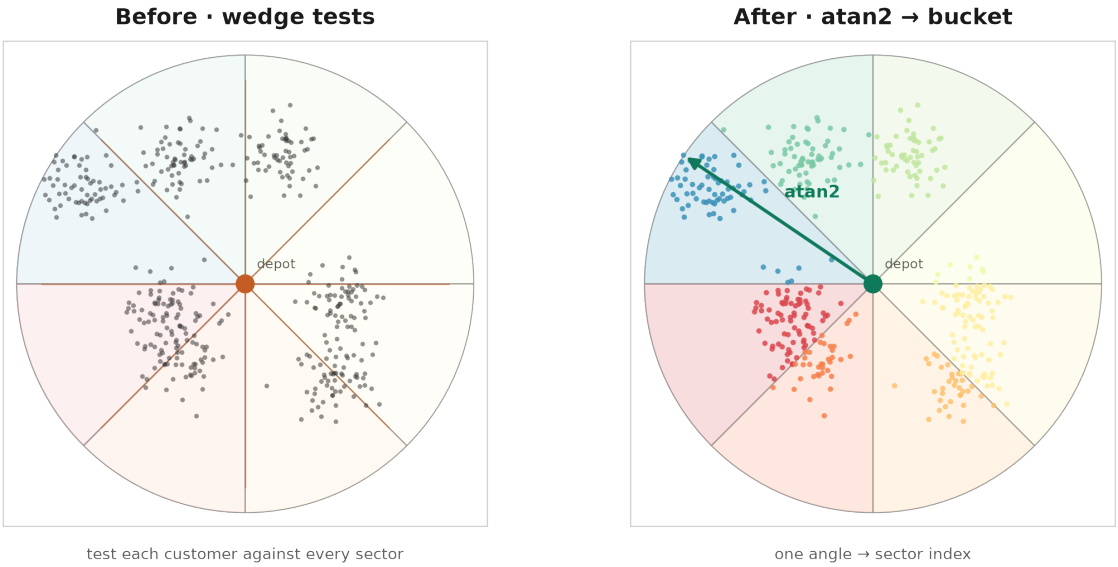


Figure 2: Depot-centered sectors on a clustered instance. Modified code maps each customer with a single angle.

2.2 M

ST and 2-opt

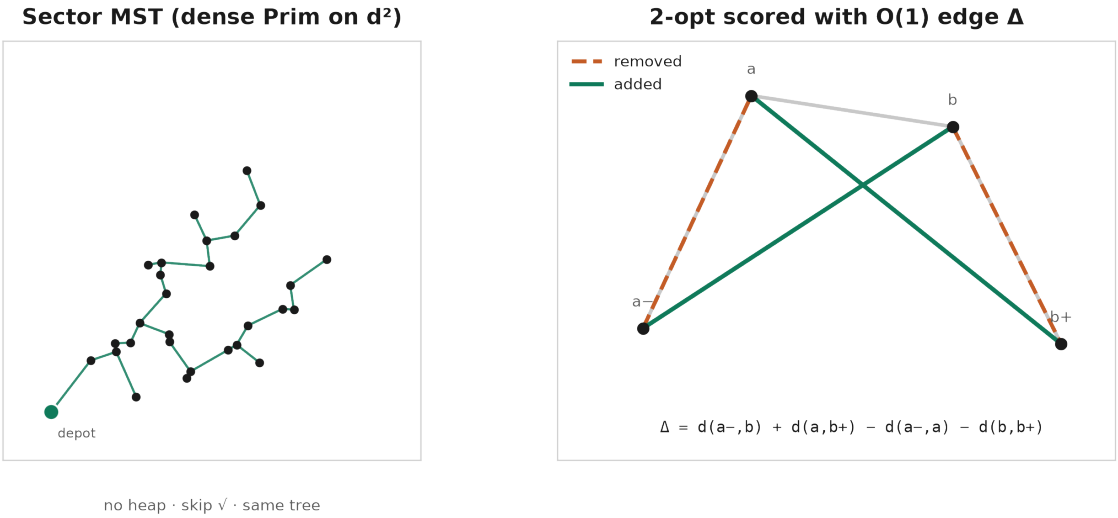


Figure 3: Left: Euclidean MST inside one sector via dense Prim on squared length. Right: a 2-opt reversal scored from the four edges that change.

2.3 P

arallel trials

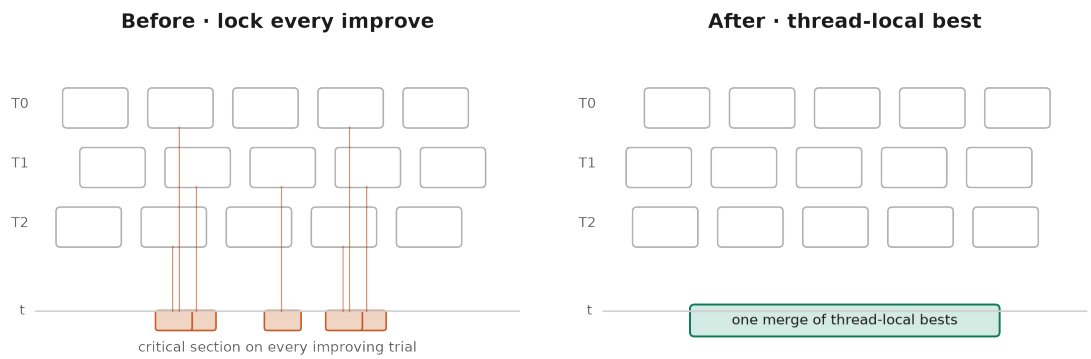


Figure 4: OpenMP over ρ trials: fewer trips through the critical section.

3 Measured results

All Main rows use synthetic clustered instances with a centered depot, α=2, ρ=10^4, 48 OpenMP threads, same machine. The timed region is `Solver::solve`.

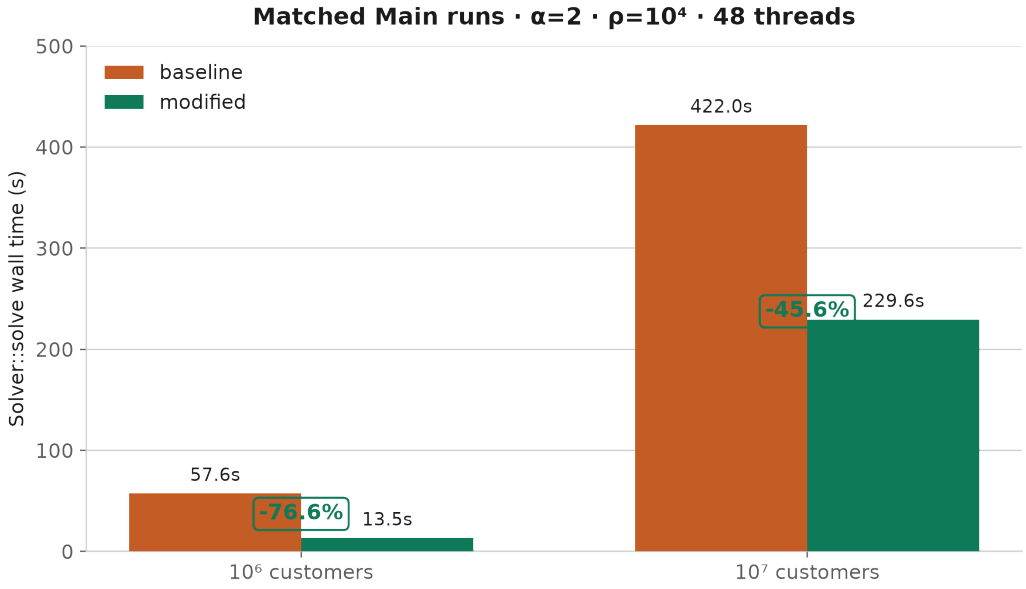


Figure 5: `Solver::solve` wall time on matched Main runs.

Customers	Baseline (s)	Modified (s)	Time change	Cost change
10^6	57.57	13.49	$\mathbf{-76.6}\%$	-0.002%
10^7	421.98	229.60	$\mathbf{-45.6}\%$	$+0.0006\%$

Table 1: Main results. Cost change is $(\text{cost}_{\text{mod}} - \text{cost}_{\text{base}})/\text{cost}_{\text{base}}$.

Stress. On a pure clustered instance with about 4.7×10^7 customers and α=5, the modified solver finished in **4.53 h** ($\approx$ 16308 s). The baseline was still inside `solve` after **more than 26 h** ($\geq$ 93600 s) and was stopped without a written solution.

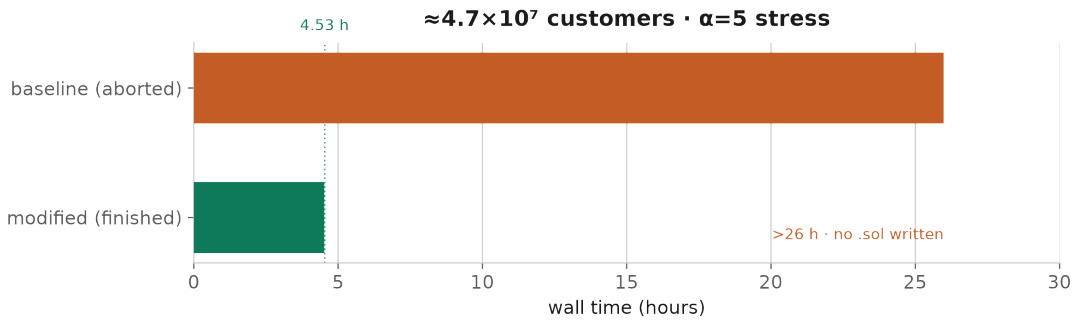


Figure 6: 47M stress run. Baseline bar is time at abort, not a finished solve.

4 Code sketches

```
// Sector index from a single angle around the depot
double deg = std::atan2(dy, dx) * 180.0 / PI;
if (deg < 0.0) deg += 360.0;
int idx = static_cast<int>(deg / alpha);
buckets[idx].push_back(u);
```

```
// Dense Prim: compare squared lengths (order matches Euclidean)
const double dw = dx * dx + dy * dy;
if (dw < best[w]) { best[w] = dw; parent[w] = v; }
```

```
// 2-opt: only the two edges changed by the reversal affect the score
const double delta =
    dist(left, b) + dist(a, right)
  - dist(left, a) - dist(b, right);
if (delta < -1e-12)
    reverse(segment from i to k);
```

The substantive edits live in `Lib/Bucket_Partitioned_MDS/Solver.cpp`, with `distance_fast` in the header and nested OpenMP setup in `Lib/Initializer.cpp`.

5 Takeaway

The published heuristic shape is kept: partition, Euclidean MST, randomized DFS, 2-opt. What changes is the cost of those steps. At one and ten million customers, wall time falls by well over 20% while solution cost stays flat; at roughly forty-seven million customers the same rewrites turn a multi-day unfinished baseline run into a finish on the order of a few hours.

Scoreboard. Rows go into a shared sheet with columns for solver (`bp-mds`, `ours-cpu`, `ours-gpu`, `cuopt`), cost, BKS gap, wall time, and hardware. CPU lane first (this note); GPU lane vs cuOpt next. Protocol: `benchmarks/` in the repo.